ISIT 225 · CLOUD COMPUTING

Event-Driven Orders with Pub/Sub

Pizza shop orders that stay safe when the kitchen side is slow

Burla Gul Celik · Muhammet Eren Osmanli
Yusuf Salih Demirbas · Graca Das Dores Salo Macaia

Prof. Tsige Tessema · Bay Atlantic University · Oct 2026

What we will cover

01 Problem & what we built

02 Architecture and why decoupling matters

03 Service contract (message format)

04 Function logic, tests, failure handling

05 Security and remaining risks

What we built

Pub/Sub topic

orders – receives every new pizza order

Cloud Function

processOrder – runs when a message arrives

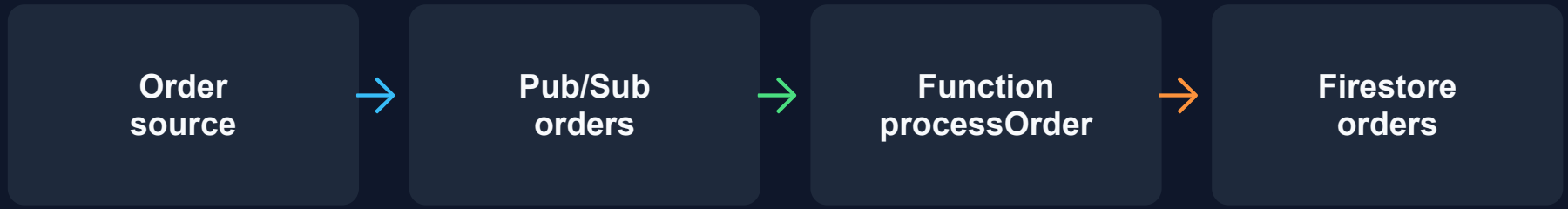
Firestore

orders collection – durable store for each order

Retries + DLQ

messages wait if the function is down; bad ones can go to dead letter

Architecture / event flow



Why decoupling helps

The order source only publishes a message. It does not wait for the kitchen or the function. If the function is offline for a minute, messages stay in the topic and Pub/Sub retries them. Both sides can scale on their own. Messages that fail many times can go to a dead-letter topic instead of disappearing.

Service contract

Every message on the orders topic is JSON. orderId is required.

```
{
  "orderId": "ord-001",
  "customerName": "Ayse",
  "items": [ { "name": "Margherita", "qty": 2 } ],
  "total": 45.50
}
```

orderId is also the Firestore document ID. If it is missing, the function throws. Pub/Sub retries a few times; after that the message can go to the dead-letter topic.

What the function does

1. Decode the Pub/Sub payload (base64 → JSON)
2. Require orderId — if missing, throw an error
3. Check if that document already exists in Firestore
4. If it exists → skip (no duplicates on retry)
5. If not → write with status "received"

Tests & failure handling

Successful path

Valid orders with different orderIds appear in Firestore with status received. Same orderId twice → second write is skipped.

Failure path

Message without orderId → function error. Pub/Sub retries; after enough failures the message can go to the dead-letter topic.

If the function is down for a minute

Messages stay in the topic. Pub/Sub keeps trying. The function only acknowledges after a successful write, so orders are not lost.

Security controls

- Dedicated service account for the function
- Least privilege: Firestore write + Pub/Sub subscribe only
- Bad messages rejected before any write
- Document ID = orderId → no duplicates on retry
- Clients do not write directly to the orders collection

Risks we are managing

1. Live demo / console issues

Test screenshots ready; we can walk through the flow without a live session.

2. Cold start (free tier)

Short delay possible; warm the function with a small test before the talk.

3. Cost

$1 budget alert set; delete topic, function, and test data after we present.

4. Time limit (5–7 min)

Short path: architecture → contract → tests → retries → security.

Summary

- Orders go through Pub/Sub so they are not lost if the kitchen is slow
- Function validates, writes once, and is safe under retries
- Decoupling lets both sides scale independently
- Least privilege + idempotent writes for security

Questions?

Burla Gul Celik · Muhammet Eren Osmanli · Yusuf Salih Demirbas · Graca Das
ISIT 225 · Prof. Tsige Tessema · Bay Atlantic University